JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class LinkedBlockingDeque<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractQueue<E>
      java.util.concurrent.LinkedBlockingDeque<E>

Type Parameters:

`E - the type of elements held in this collection`

All Implemented Interfaces:

`Serializable, Iterable<E>, Collection<E>, BlockingDeque<E>, BlockingQueue<E>, Deque<E>, Queue<E>`

```
public class LinkedBlockingDeque<E>
extends AbstractQueue<E>
implements BlockingDeque<E>, Serializable
```

An optionally-bounded blocking deque based on linked nodes.

The optional capacity bound constructor argument serves as a way to prevent excessive expansion. The capacity, if unspecified, is equal to `Integer.MAX_VALUE`. Linked nodes are dynamically created upon each insertion unless this would bring the deque above capacity.

Most operations run in constant time (ignoring time spent blocking). Exceptions include `remove`, `removeFirstOccurrence`, `removeLastOccurrence`, `contains`, `iterator.remove()`, and the bulk operations, all of which run in linear time.

This class and its iterator implement all of the *optional* methods of the `Collection` and `Iterator` interfaces.

This class is a member of the Java Collections Framework.

Since:

`1.6`

See Also:

Serialized Form

Constructor Summary

Constructors

Constructor	Description
`LinkedBlockingDeque()`	Creates a `LinkedBlockingDeque` with a capacity of `Integer.MAX_VALUE`.
`LinkedBlockingDeque(Collection<? extends E> c)`	Creates a `LinkedBlockingDeque` with a capacity of `Integer.MAX_VALUE`, initially containing the elements of the given collection, added in traversal order of the collection's iterator.
`LinkedBlockingDeque(int capacity)`	Creates a `LinkedBlockingDeque` with the given (fixed) capacity.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`boolean`	`add(E e)`	Inserts the specified element at the end of this deque unless it would violate capacity restrictions.
`void`	`addFirst(E e)`	Inserts the specified element at the front of this deque if it is possible to do so immediately without

		violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
void	**addLast**(**E** e)	Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
void	**clear**()	Atomically removes all of the elements from this deque.
boolean	**contains**(**Object** o)	Returns `true` if this deque contains the specified element.
Iterator<**E**>	**descendingIterator**()	Returns an iterator over the elements in this deque in reverse sequential order.
int	**drainTo**(**Collection**<? super **E**> c)	Removes all available elements from this queue and adds them to the given collection.
int	**drainTo**(**Collection**<? super **E**> c, int maxElements)	Removes at most the given number of available elements from this queue and adds them to the given collection.
E	**element**()	Retrieves, but does not remove, the head of the queue represented by this deque.
E	**getFirst**()	

		Retrieves, but does not remove, the first element of this deque.
E	**getLast**()	Retrieves, but does not remove, the last element of this deque.
Iterator<**E**>	**iterator**()	Returns an iterator over the elements in this deque in proper sequence.
boolean	**offer**(**E** e)	Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque) if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and `false` if no space is currently available.
boolean	**offer**(**E** e, long timeout, **TimeUnit** unit)	Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque), waiting up to the specified wait time if necessary for space to become available.
boolean	**offerFirst**(**E** e)	Inserts the specified element at the front of this deque if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and `false` if no space is currently available.
boolean	**offerFirst**(**E** e, long timeout, **TimeUnit** unit)	Inserts the specified element at the front of

		this deque, waiting up to the specified wait time if necessary for space to become available.
boolean	**offerLast**(**E** e)	Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and `false` if no space is currently available.
boolean	**offerLast**(**E** e, long timeout, **TimeUnit** unit)	Inserts the specified element at the end of this deque, waiting up to the specified wait time if necessary for space to become available.
E	**peek**()	Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.
E	**peekFirst**()	Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.
E	**peekLast**()	Retrieves, but does not remove, the last element of this deque, or returns `null` if this deque is empty.
E	**poll**()	Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this

		deque), or returns `null` if this deque is empty.
E	**poll**(long timeout, **TimeUnit** unit)	Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), waiting up to the specified wait time if necessary for an element to become available.
E	**pollFirst**()	Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.
E	**pollFirst**(long timeout, **TimeUnit** unit)	Retrieves and removes the first element of this deque, waiting up to the specified wait time if necessary for an element to become available.
E	**pollLast**()	Retrieves and removes the last element of this deque, or returns `null` if this deque is empty.
E	**pollLast**(long timeout, **TimeUnit** unit)	Retrieves and removes the last element of this deque, waiting up to the specified wait time if necessary for an element to become available.
E	**pop**()	Pops an element from the stack represented by this deque.
void	**push**(**E** e)	Pushes an element onto the stack represented by this deque (in other words, at the head of this deque) if it is possible to do so

		immediately without violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.
void	**put**(**E** e)	Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque), waiting if necessary for space to become available.
void	**putFirst**(**E** e)	Inserts the specified element at the front of this deque, waiting if necessary for space to become available.
void	**putLast**(**E** e)	Inserts the specified element at the end of this deque, waiting if necessary for space to become available.
int	**remainingCapacity**()	Returns the number of additional elements that this deque can ideally (in the absence of memory or resource constraints) accept without blocking.
E	**remove**()	Retrieves and removes the head of the queue represented by this deque.
boolean	**remove**(**Object** o)	Removes the first occurrence of the specified element from this deque.

E	**removeFirst**()	Retrieves and removes the first element of this deque.
boolean	**removeFirstOccurrence**(**Object** o)	Removes the first occurrence of the specified element from this deque.
E	**removeLast**()	Retrieves and removes the last element of this deque.
boolean	**removeLastOccurrence**(**Object** o)	Removes the last occurrence of the specified element from this deque.
int	**size**()	Returns the number of elements in this deque.
Spliterator<**E**>	**spliterator**()	Returns a **Spliterator** over the elements in this deque.
E	**take**()	Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), waiting if necessary until an element becomes available.
E	**takeFirst**()	Retrieves and removes the first element of this deque, waiting if necessary until an element becomes available.
E	**takeLast**()	Retrieves and removes the last element of this deque, waiting if necessary until an element becomes available.

Object[]	toArray()	Returns an array containing all of the elements in this deque, in proper sequence (from first to last element).
<T> T[]	toArray(T[] a)	Returns an array containing all of the elements in this deque, in proper sequence; the runtime type of the returned array is that of the specified array.
String	toString()	Returns a string representation of this collection.

Methods inherited from class java.util.AbstractQueue

addAll

Methods inherited from class java.util.AbstractCollection

containsAll, isEmpty, removeAll, retainAll

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Collection

addAll, containsAll, equals, hashCode, isEmpty, parallelStream, removeAll, removeIf, retainAll, stream

Methods inherited from interface java.lang.Iterable

forEach

Constructor Detail

LinkedBlockingDeque

```
public LinkedBlockingDeque()
```

Creates a `LinkedBlockingDeque` with a capacity of `Integer.MAX_VALUE`.

LinkedBlockingDeque

```
public LinkedBlockingDeque(int capacity)
```

Creates a `LinkedBlockingDeque` with the given (fixed) capacity.

Parameters:

`capacity` - the capacity of this deque

Throws:

`IllegalArgumentException` - if capacity is less than 1

LinkedBlockingDeque

```
public LinkedBlockingDeque(Collection<? extends E> c)
```

Creates a `LinkedBlockingDeque` with a capacity of `Integer.MAX_VALUE`, initially containing the elements of the given collection, added in traversal order of the collection's iterator.

Parameters:

`c` - the collection of elements to initially contain

Throws:

`NullPointerException` - if the specified collection or any of its elements are null

Method Detail

addFirst

```
public void addFirst(E e)
```

Description copied from interface: `BlockingDeque`

Inserts the specified element at the front of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an

IllegalStateException if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use offerFirst.

Specified by:

addFirst in interface BlockingDeque<E>

Specified by:

addFirst in interface Deque<E>

Parameters:

e - the element to add

Throws:

IllegalStateException - if this deque is full

NullPointerException - if the specified element is null

addLast

public void addLast(E e)

Description copied from interface: BlockingDeque

Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, throwing an IllegalStateException if no space is currently available. When using a capacity-restricted deque, it is generally preferable to use offerLast.

Specified by:

addLast in interface BlockingDeque<E>

Specified by:

addLast in interface Deque<E>

Parameters:

e - the element to add

Throws:

IllegalStateException - if this deque is full

NullPointerException - if the specified element is null

offerFirst

public boolean offerFirst(E e)

Description copied from interface: BlockingDeque

Inserts the specified element at the front of this deque if it is possible to do so immediately without violating capacity restrictions, returning true upon success and false if no space is currently available. When using a capacity-

restricted deque, this method is generally preferable to the addFirst method, which can fail to insert an element only by throwing an exception.

Specified by:

offerFirst in interface BlockingDeque<E>

Specified by:

offerFirst in interface Deque<E>

Parameters:

e - the element to add

Returns:

true if the element was added to this deque, else false

Throws:

NullPointerException - if the specified element is null

offerLast

```
public boolean offerLast(E e)
```

Description copied from interface: BlockingDeque

Inserts the specified element at the end of this deque if it is possible to do so immediately without violating capacity restrictions, returning true upon success and false if no space is currently available. When using a capacity-restricted deque, this method is generally preferable to the addLast method, which can fail to insert an element only by throwing an exception.

Specified by:

offerLast in interface BlockingDeque<E>

Specified by:

offerLast in interface Deque<E>

Parameters:

e - the element to add

Returns:

true if the element was added to this deque, else false

Throws:

NullPointerException - if the specified element is null

putFirst

```
public void putFirst(E e)
              throws InterruptedException
```

Description copied from interface: BlockingDeque

Inserts the specified element at the front of this deque, waiting if necessary for space to become available.

Specified by:

`putFirst` in interface `BlockingDeque<E>`

Parameters:

`e` - the element to add

Throws:

`NullPointerException` - if the specified element is null

`InterruptedException` - if interrupted while waiting

putLast

```
public void putLast(E e)
             throws InterruptedException
```

Description copied from interface: `BlockingDeque`

Inserts the specified element at the end of this deque, waiting if necessary for space to become available.

Specified by:

`putLast` in interface `BlockingDeque<E>`

Parameters:

`e` - the element to add

Throws:

`NullPointerException` - if the specified element is null

`InterruptedException` - if interrupted while waiting

offerFirst

```
public boolean offerFirst(E e,
                          long timeout,
                          TimeUnit unit)
                   throws InterruptedException
```

Description copied from interface: `BlockingDeque`

Inserts the specified element at the front of this deque, waiting up to the specified wait time if necessary for space to become available.

Specified by:

`offerFirst` in interface `BlockingDeque<E>`

Parameters:

`e` - the element to add

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

true if successful, or false if the specified waiting time elapses before space is available

Throws:

NullPointerException - if the specified element is null

InterruptedException - if interrupted while waiting

offerLast

```
public boolean offerLast(E e,
                         long timeout,
                         TimeUnit unit)
                  throws InterruptedException
```

Description copied from interface: BlockingDeque

Inserts the specified element at the end of this deque, waiting up to the specified wait time if necessary for space to become available.

Specified by:

offerLast in interface BlockingDeque<E>

Parameters:

e - the element to add

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

true if successful, or false if the specified waiting time elapses before space is available

Throws:

NullPointerException - if the specified element is null

InterruptedException - if interrupted while waiting

removeFirst

```
public E removeFirst()
```

Description copied from interface: Deque

Retrieves and removes the first element of this deque. This method differs from `pollFirst` only in that it throws an exception if this deque is empty.

Specified by:

`removeFirst` in interface `Deque<E>`

Returns:

the head of this deque

Throws:

`NoSuchElementException` - if this deque is empty

removeLast

`public E removeLast()`

Description copied from interface: Deque

Retrieves and removes the last element of this deque. This method differs from `pollLast` only in that it throws an exception if this deque is empty.

Specified by:

`removeLast` in interface `Deque<E>`

Returns:

the tail of this deque

Throws:

`NoSuchElementException` - if this deque is empty

pollFirst

`public E pollFirst()`

Description copied from interface: Deque

Retrieves and removes the first element of this deque, or returns `null` if this deque is empty.

Specified by:

`pollFirst` in interface `Deque<E>`

Returns:

the head of this deque, or null if this deque is empty

pollLast

`public E pollLast()`

Description copied from interface: Deque

Retrieves and removes the last element of this deque, or returns `null` if this deque is empty.

Specified by:

`pollLast` in interface `Deque<E>`

Returns:

`the tail of this deque, or null if this deque is empty`

takeFirst

```
public E takeFirst()
            throws InterruptedException
```

Description copied from interface: `BlockingDeque`

Retrieves and removes the first element of this deque, waiting if necessary until an element becomes available.

Specified by:

`takeFirst` in interface `BlockingDeque<E>`

Returns:

`the head of this deque`

Throws:

`InterruptedException - if interrupted while waiting`

takeLast

```
public E takeLast()
           throws InterruptedException
```

Description copied from interface: `BlockingDeque`

Retrieves and removes the last element of this deque, waiting if necessary until an element becomes available.

Specified by:

`takeLast` in interface `BlockingDeque<E>`

Returns:

`the tail of this deque`

Throws:

`InterruptedException - if interrupted while waiting`

pollFirst

```
public E pollFirst(long timeout,
                   TimeUnit unit)
            throws InterruptedException
```

Description copied from interface: BlockingDeque

Retrieves and removes the first element of this deque, waiting up to the specified wait time if necessary for an element to become available.

Specified by:

`pollFirst` in interface `BlockingDeque<E>`

Parameters:

`timeout` - how long to wait before giving up, in units of `unit`

`unit` - a TimeUnit determining how to interpret the `timeout` parameter

Returns:

the head of this deque, or null if the specified waiting time elapses before an element is available

Throws:

`InterruptedException` - if interrupted while waiting

pollLast

```
public E pollLast(long timeout,
                  TimeUnit unit)
           throws InterruptedException
```

Description copied from interface: BlockingDeque

Retrieves and removes the last element of this deque, waiting up to the specified wait time if necessary for an element to become available.

Specified by:

pollLast in interface BlockingDeque<E>

Parameters:

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

the tail of this deque, or null if the specified waiting time elapses before an element is available

Throws:

InterruptedException - if interrupted while waiting

getFirst

```
public E getFirst()
```

Description copied from interface: Deque

Retrieves, but does not remove, the first element of this deque. This method differs from peekFirst only in that it throws an exception if this deque is empty.

Specified by:

getFirst in interface Deque<E>

Returns:

the head of this deque

Throws:

NoSuchElementException - if this deque is empty

getLast

```
public E getLast()
```

Description copied from interface: Deque

Retrieves, but does not remove, the last element of this deque. This method differs from `peekLast` only in that it throws an exception if this deque is empty.

Specified by:

`getLast` in interface `Deque<E>`

Returns:

the tail of this deque

Throws:

`NoSuchElementException` - if this deque is empty

peekFirst

```
public E peekFirst()
```

Description copied from interface: Deque

Retrieves, but does not remove, the first element of this deque, or returns `null` if this deque is empty.

Specified by:

`peekFirst` in interface `Deque<E>`

Returns:

the head of this deque, or null if this deque is empty

peekLast

```
public E peekLast()
```

Description copied from interface: Deque

Retrieves, but does not remove, the last element of this deque, or returns `null` if this deque is empty.

Specified by:

`peekLast` in interface `Deque<E>`

Returns:

the tail of this deque, or null if this deque is empty

removeFirstOccurrence

```
public boolean removeFirstOccurrence(Object o)
```

Description copied from interface: BlockingDeque

Removes the first occurrence of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes

the first element e such that `o.equals(e)` (if such an element exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

Specified by:

`removeFirstOccurrence` in interface `BlockingDeque<E>`

Specified by:

`removeFirstOccurrence` in interface `Deque<E>`

Parameters:

`o - element to be removed from this deque, if present`

Returns:

`true if an element was removed as a result of this call`

removeLastOccurrence

`public boolean removeLastOccurrence(Object o)`

Description copied from interface: `BlockingDeque`

Removes the last occurrence of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes the last element e such that `o.equals(e)` (if such an element exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

Specified by:

`removeLastOccurrence` in interface `BlockingDeque<E>`

Specified by:

`removeLastOccurrence` in interface `Deque<E>`

Parameters:

`o - element to be removed from this deque, if present`

Returns:

`true if an element was removed as a result of this call`

add

`public boolean add(E e)`

Inserts the specified element at the end of this deque unless it would violate capacity restrictions. When using a capacity-restricted deque, it is generally preferable to use method `offer`.

This method is equivalent to `addLast(E)`.

Specified by:

add in interface Collection<E>

Specified by:

add in interface BlockingDeque<E>

Specified by:

add in interface BlockingQueue<E>

Specified by:

add in interface Deque<E>

Specified by:

add in interface Queue<E>

Overrides:

add in class AbstractQueue<E>

Parameters:

e - the element to add

Returns:

true (as specified by Collection.add(E))

Throws:

IllegalStateException - if this deque is full

NullPointerException - if the specified element is null

offer

```
public boolean offer(E e)
```

Description copied from interface: BlockingDeque

Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque) if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and `false` if no space is currently available. When using a capacity-restricted deque, this method is generally preferable to the BlockingDeque.add(E) method, which can fail to insert an element only by throwing an exception.

This method is equivalent to `offerLast`.

Specified by:

offer in interface BlockingDeque<E>

Specified by:

offer in interface BlockingQueue<E>

Specified by:

offer in interface Deque<E>

Specified by:

offer in interface Queue<E>

Parameters:

`e - the element to add`

Returns:

`true if the element was added to this queue, else false`

Throws:

`NullPointerException - if the specified element is null`

put

```
public void put(E e)
         throws InterruptedException
```

Description copied from interface: BlockingDeque

Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque), waiting if necessary for space to become available.

This method is equivalent to `putLast`.

Specified by:

`put in interface BlockingDeque<E>`

Specified by:

`put in interface BlockingQueue<E>`

Parameters:

`e - the element to add`

Throws:

`NullPointerException - if the specified element is null`

`InterruptedException - if interrupted while waiting`

offer

```
public boolean offer(E e,
                     long timeout,
                     TimeUnit unit)
              throws InterruptedException
```

Description copied from interface: BlockingDeque

Inserts the specified element into the queue represented by this deque (in other words, at the tail of this deque), waiting up to the specified wait time if necessary for space to become available.

This method is equivalent to `offerLast`.

Specified by:

offer in interface BlockingDeque<E>

Specified by:

offer in interface BlockingQueue<E>

Parameters:

e - the element to add

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

true if the element was added to this deque, else false

Throws:

NullPointerException - if the specified element is null

InterruptedException - if interrupted while waiting

remove

public E remove()

Retrieves and removes the head of the queue represented by this deque. This method differs from poll only in that it throws an exception if this deque is empty.

This method is equivalent to removeFirst.

Specified by:

remove in interface BlockingDeque<E>

Specified by:

remove in interface Deque<E>

Specified by:

remove in interface Queue<E>

Overrides:

remove in class AbstractQueue<E>

Returns:

the head of the queue represented by this deque

Throws:

NoSuchElementException - if this deque is empty

poll

```
public E poll()
```

Description copied from interface: BlockingDeque

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `Deque.pollFirst()`.

Specified by:

`poll` in interface `BlockingDeque<E>`

Specified by:

`poll` in interface `Deque<E>`

Specified by:

`poll` in interface `Queue<E>`

Returns:

the head of this deque, or null if this deque is empty

take

```
public E take()
       throws InterruptedException
```

Description copied from interface: BlockingDeque

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), waiting if necessary until an element becomes available.

This method is equivalent to `takeFirst`.

Specified by:

`take` in interface `BlockingDeque<E>`

Specified by:

`take` in interface `BlockingQueue<E>`

Returns:

the head of this deque

Throws:

`InterruptedException` - if interrupted while waiting

poll

```
public E poll(long timeout,
              TimeUnit unit)
       throws InterruptedException
```

Description copied from interface: BlockingDeque

Retrieves and removes the head of the queue represented by this deque (in other words, the first element of this deque), waiting up to the specified wait time if necessary for an element to become available.

This method is equivalent to pollFirst.

Specified by:

poll in interface BlockingDeque<E>

Specified by:

poll in interface BlockingQueue<E>

Parameters:

timeout - how long to wait before giving up, in units of unit

unit - a TimeUnit determining how to interpret the timeout parameter

Returns:

the head of this deque, or null if the specified waiting time elapses before an element is available

Throws:

InterruptedException - if interrupted while waiting

element

```
public E element()
```

Retrieves, but does not remove, the head of the queue represented by this deque. This method differs from peek only in that it throws an exception if this deque is empty.

This method is equivalent to getFirst.

Specified by:

element in interface BlockingDeque<E>

Specified by:

element in interface Deque<E>

Specified by:

element in interface Queue<E>

Overrides:

`element` in class `AbstractQueue<E>`

Returns:

the head of the queue represented by this deque

Throws:

`NoSuchElementException` - if this deque is empty

peek

```
public E peek()
```

Description copied from interface: `BlockingDeque`

Retrieves, but does not remove, the head of the queue represented by this deque (in other words, the first element of this deque), or returns `null` if this deque is empty.

This method is equivalent to `peekFirst`.

Specified by:

`peek` in interface `BlockingDeque<E>`

Specified by:

`peek` in interface `Deque<E>`

Specified by:

`peek` in interface `Queue<E>`

Returns:

the head of this deque, or null if this deque is empty

remainingCapacity

```
public int remainingCapacity()
```

Returns the number of additional elements that this deque can ideally (in the absence of memory or resource constraints) accept without blocking. This is always equal to the initial capacity of this deque less the current `size` of this deque.

Note that you *cannot* always tell if an attempt to insert an element will succeed by inspecting `remainingCapacity` because it may be the case that another thread is about to insert or remove an element.

Specified by:

`remainingCapacity` in interface `BlockingQueue<E>`

Returns:

the remaining capacity

drainTo

```
public int drainTo(Collection<? super E> c)
```

Description copied from interface: `BlockingQueue`

Removes all available elements from this queue and adds them to the given collection. This operation may be more efficient than repeatedly polling this queue. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

`drainTo` in interface `BlockingQueue<E>`

Parameters:

`c` - the collection to transfer elements into

Returns:

the number of elements transferred

Throws:

`UnsupportedOperationException` - if addition of elements is not supported by the specified collection

`ClassCastException` - if the class of an element of this queue prevents it from being added to the specified collection

`NullPointerException` - if the specified collection is null

IllegalArgumentException - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

drainTo

```
public int drainTo(Collection<? super E> c,
                   int maxElements)
```

Description copied from interface: BlockingQueue

Removes at most the given number of available elements from this queue and adds them to the given collection. A failure encountered while attempting to add elements to collection `c` may result in elements being in neither, either or both collections when the associated exception is thrown. Attempts to drain a queue to itself result in `IllegalArgumentException`. Further, the behavior of this operation is undefined if the specified collection is modified while the operation is in progress.

Specified by:

drainTo in interface BlockingQueue<E>

Parameters:

c - the collection to transfer elements into

maxElements - the maximum number of elements to transfer

Returns:

the number of elements transferred

Throws:

UnsupportedOperationException - if addition of elements is not supported by the specified collection

ClassCastException - if the class of an element of this queue prevents it from being added to the specified collection

NullPointerException - if the specified collection is null

IllegalArgumentException - if the specified collection is this queue, or some property of an element of this queue prevents it from being added to the specified collection

push

```
public void push(E e)
```

Description copied from interface: BlockingDeque

Pushes an element onto the stack represented by this deque (in other words, at the head of this deque) if it is possible to do so immediately without

violating capacity restrictions, throwing an `IllegalStateException` if no space is currently available.

This method is equivalent to `addFirst`.

Specified by:

`push` in interface `BlockingDeque<E>`

Specified by:

`push` in interface `Deque<E>`

Parameters:

`e` - the element to push

Throws:

`IllegalStateException` - if this deque is full

`NullPointerException` - if the specified element is null

pop

```
public E pop()
```

Description copied from interface: Deque

Pops an element from the stack represented by this deque. In other words, removes and returns the first element of this deque.

This method is equivalent to `Deque.removeFirst()`.

Specified by:

`pop` in interface `Deque<E>`

Returns:

the element at the front of this deque (which is the top of the stack represented by this deque)

Throws:

`NoSuchElementException` - if this deque is empty

remove

```
public boolean remove(Object o)
```

Removes the first occurrence of the specified element from this deque. If the deque does not contain the element, it is unchanged. More formally, removes the first element `e` such that `o.equals(e)` (if such an element exists). Returns `true` if this deque contained the specified element (or equivalently, if this deque changed as a result of the call).

This method is equivalent to `removeFirstOccurrence`.

Specified by:

`remove` in interface `Collection<E>`

Specified by:

`remove` in interface `BlockingDeque<E>`

Specified by:

`remove` in interface `BlockingQueue<E>`

Specified by:

`remove` in interface `Deque<E>`

Overrides:

`remove` in class `AbstractCollection<E>`

Parameters:

`o` - element to be removed from this deque, if present

Returns:

`true` if this deque changed as a result of the call

size

```
public int size()
```

Returns the number of elements in this deque.

Specified by:

`size` in interface `Collection<E>`

Specified by:

`size` in interface `BlockingDeque<E>`

Specified by:

`size` in interface `Deque<E>`

Specified by:

`size` in class `AbstractCollection<E>`

Returns:

the number of elements in this deque

contains

```
public boolean contains(Object o)
```

Returns `true` if this deque contains the specified element. More formally, returns `true` if and only if this deque contains at least one element `e` such that `o.equals(e)`.

Specified by:

contains in interface Collection<E>

Specified by:

contains in interface BlockingDeque<E>

Specified by:

contains in interface BlockingQueue<E>

Specified by:

contains in interface Deque<E>

Overrides:

contains in class AbstractCollection<E>

Parameters:

o - object to be checked for containment in this deque

Returns:

true if this deque contains the specified element

toArray

public Object[] toArray()

Returns an array containing all of the elements in this deque, in proper sequence (from first to last element).

The returned array will be "safe" in that no references to it are maintained by this deque. (In other words, this method must allocate a new array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

toArray in interface Collection<E>

Overrides:

toArray in class AbstractCollection<E>

Returns:

an array containing all of the elements in this deque

toArray

public <T> T[] toArray(T[] a)

Returns an array containing all of the elements in this deque, in proper sequence; the runtime type of the returned array is that of the specified array. If the deque fits in the specified array, it is returned therein.

Otherwise, a new array is allocated with the runtime type of the specified array and the size of this deque.

If this deque fits in the specified array with room to spare (i.e., the array has more elements than this deque), the element in the array immediately following the end of the deque is set to `null`.

Like the `toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a deque known to contain only strings. The following code can be used to dump the deque into a newly allocated array of `String`:

```
String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Overrides:

`toArray` in class `AbstractCollection<E>`

Type Parameters:

`T` - the runtime type of the array to contain the collection

Parameters:

`a` - the array into which the elements of the deque are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose

Returns:

an array containing all of the elements in this deque

Throws:

`ArrayStoreException` - if the runtime type of the specified array is not a supertype of the runtime type of every element in this deque

`NullPointerException` - if the specified array is null

toString

```
public String toString()
```

Description copied from class: `AbstractCollection`

Returns a string representation of this collection. The string representation consists of a list of the collection's elements in the order they are returned by its iterator, enclosed in square brackets ("[]"). Adjacent elements are

separated by the characters ", " (comma and space). Elements are converted to strings as by `String.valueOf(Object)`.

Overrides:

`toString` in class `AbstractCollection<E>`

Returns:

a string representation of this collection

clear

```
public void clear()
```

Atomically removes all of the elements from this deque. The deque will be empty after this call returns.

Specified by:

`clear` in interface `Collection<E>`

Overrides:

`clear` in class `AbstractQueue<E>`

iterator

```
public Iterator<E> iterator()
```

Returns an iterator over the elements in this deque in proper sequence. The elements will be returned in order from first (head) to last (tail).

The returned iterator is *weakly consistent*.

Specified by:

`iterator` in interface `Iterable<E>`

Specified by:

`iterator` in interface `Collection<E>`

Specified by:

`iterator` in interface `BlockingDeque<E>`

Specified by:

`iterator` in interface `Deque<E>`

Specified by:

`iterator` in class `AbstractCollection<E>`

Returns:

an iterator over the elements in this deque in proper sequence

descendingIterator

```
public Iterator<E> descendingIterator()
```

Returns an iterator over the elements in this deque in reverse sequential order. The elements will be returned in order from last (tail) to first (head).

The returned iterator is *weakly consistent*.

Specified by:

`descendingIterator` in interface `Deque<E>`

Returns:

an iterator over the elements in this deque in reverse order

spliterator

```
public Spliterator<E> spliterator()
```

Returns a `Spliterator` over the elements in this deque.

The returned spliterator is *weakly consistent*.

The `Spliterator` reports `Spliterator.CONCURRENT`, `Spliterator.ORDERED`, and `Spliterator.NONNULL`.

Specified by:

`spliterator` in interface `Iterable<E>`

Specified by:

`spliterator` in interface `Collection<E>`

Implementation Note:

The Spliterator implements trySplit to permit limited parallelism.

Returns:

a Spliterator over the elements in this deque

Since:

```
1.8
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.